OOPs - The Big Picture

Why do we need Object-Oriented Programming?

Object-oriented programming was developed because limitations were discovered in earlier approaches to programming.

A program in a procedural language is a list of instructions.

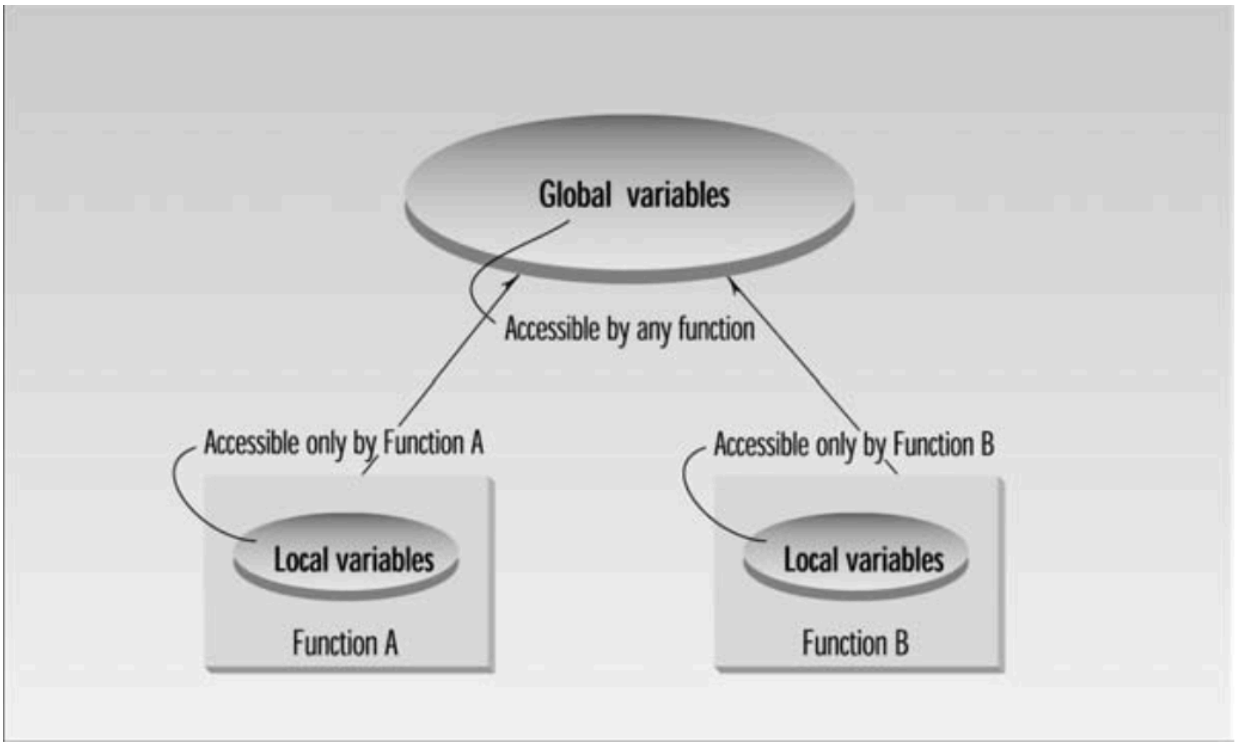
Procedural language - C, Pascal, FORTON ,etc

Problems with Structured Programming

Analyzing the reasons for these failures reveals that there are weaknesses in the procedural paradigm itself. No matter how well the structured programming approach is implemented, large programs become excessively complex.

What are the reasons for these problems with procedural languages? There are two related problems. First, functions have unrestricted access to global data. Second, unrelated functions and data, the basis of the procedural paradigm, provide a poor model of the real world

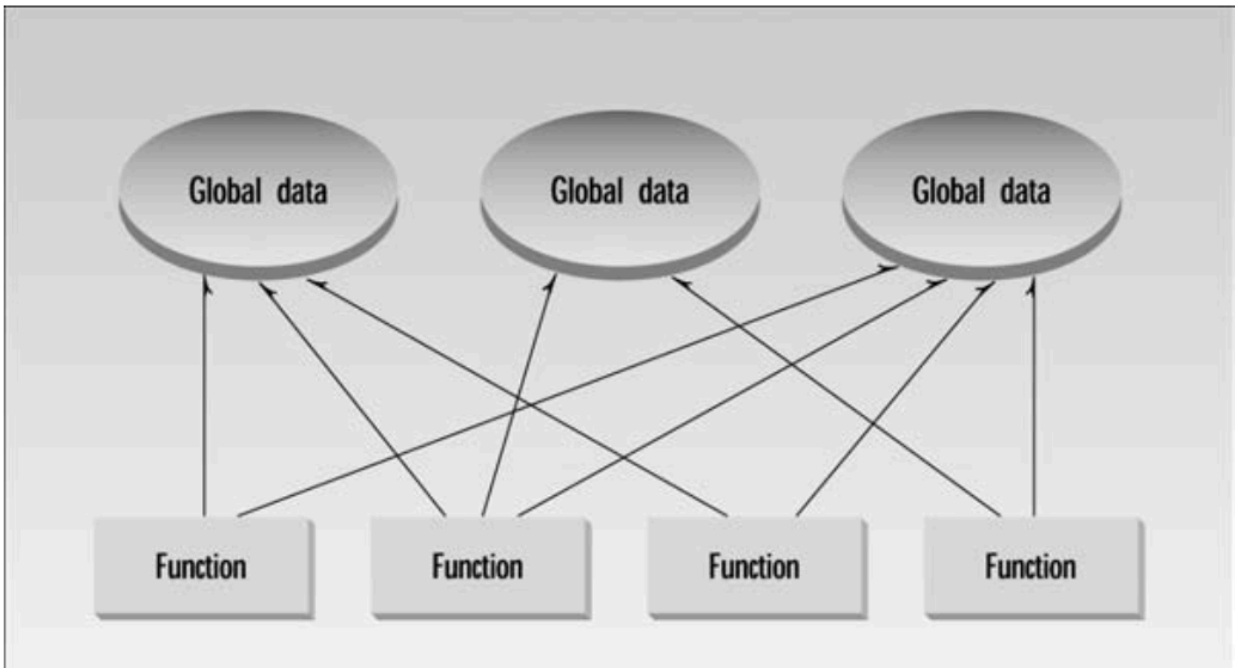
Unrestricted Access



This large number of connections causes problems in several ways.

First, it makes a program's structure difficult to conceptualize.

Second, it makes the program difficult to modify. A change made in a global data item may necessitate rewriting all the functions that access that item.



Real-World Modeling

The second—and more important—problem with the procedural paradigm is that its arrangement of separate data and functions does a poor job of modeling things in the
real world.

In the physical world we deal with objects such as people and cars.

Such objects aren't like data and they aren't like functions.

Complex real-world objects have both *attributes* and *behavior*.

Attributes

sometimes called *characteristics*

Attributes in the real world are equivalent to data in a program: they have a certain specific values, such as blue (for eye color) or four (for the number of doors).

Behavior

Behavior is something a real-world object does in response to some stimulus

Behavior is like a function: you call a function to do something (display the inventory, for example) and it does it.

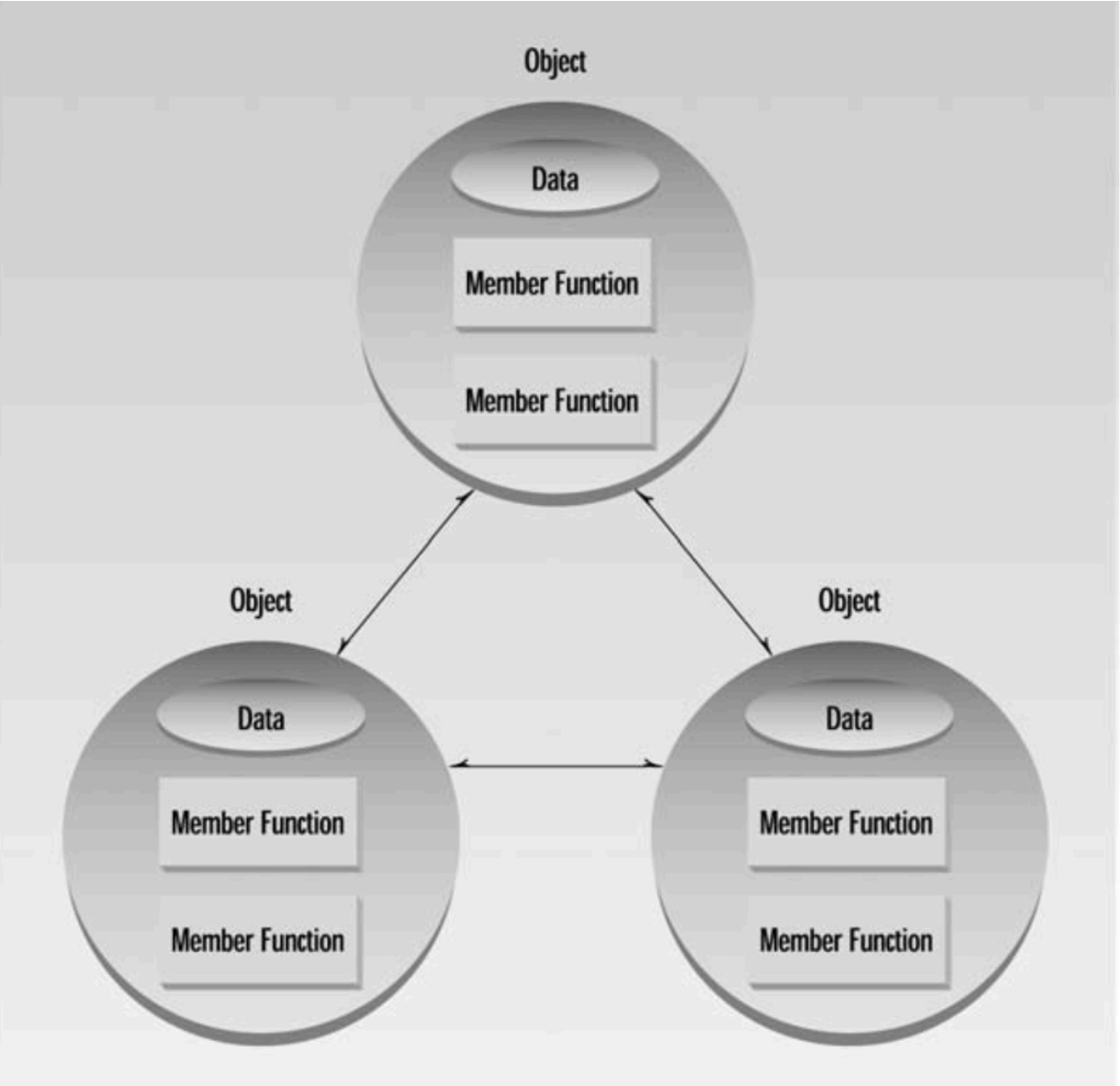
So neither data nor functions, by themselves, model real-world objects effectively.

The Object-Oriented Approach

The fundamental idea behind object-oriented languages is to combine into a single unit both *data* and the *functions that operate on that data*. Such a unit is called an *object*.

An object's functions, called *member functions* in C++, typically provide the only way to access its data. If you want to read a data item in an object, you call a member function in the object. It will access the data and return the value to you. You can't access the data directly.

The data is *hidden*, so it is safe from accidental alteration. Data and its functions are said to be *encapsulated* into a single entity. *Data encapsulation* and *data hiding* are key terms in the description of object-oriented languages.



Characteristics of Object-Oriented Languages

Objects

Object can be any real world entity

This results from the close match between objects in the programming sense and objects in the real world.

Classes

In OOP we say that objects are members of *classes*

You can define many objects of the same class,

A class serves as a plan, or blueprint. It specifies what data and what functions will be included in objects of that class. Defining the class doesn't create any objects, just as the mere existence of data type int doesn't create any variables.

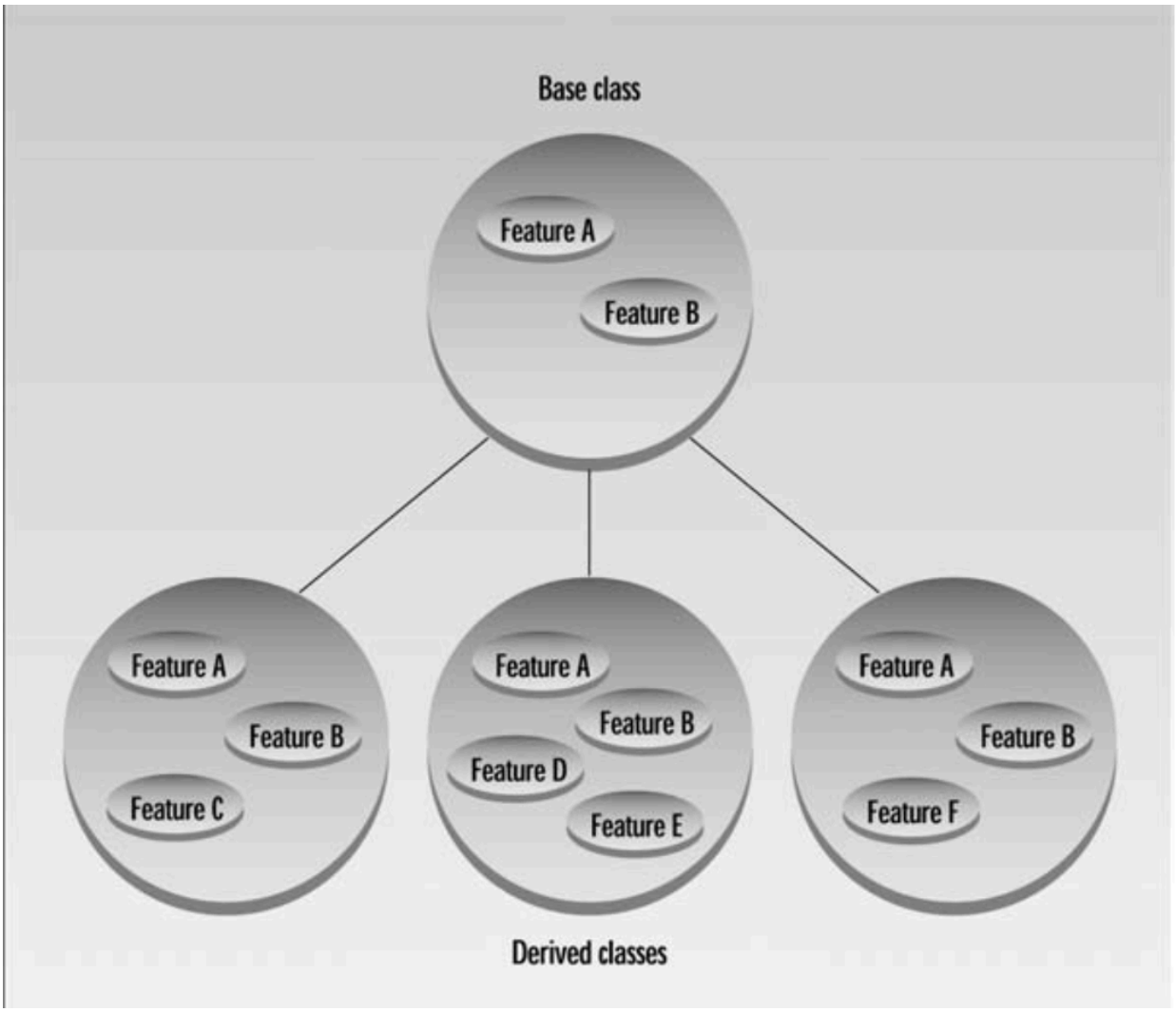
A class is thus a description of a number of similar objects. This fits our non-technical understanding of the word *class*.

An object is often called an "instance" of a class.

Inheritance

In a similar way, an OOP class can become a parent of several subclasses. In C++ the original
class is called the *base class*; other classes can be defined that share its

characteristics, but add
their own as well. These are called *derived classes*.



Inheritance is somewhat analogous to using functions to simplify a traditional procedural pro- gram. If we find that three different sections of a procedural program do almost exactly the same thing, we recognize an opportunity to extract the common elements of these three sec- tions and put them into a single function. The three sections of the program can call the func- tion to execute the common actions, and they can perform their own individual processing as well. Similarly, a base class contains elements common to a group of derived classes.

As func- tions do in a procedural program, inheritance shortens an object-oriented program and clarifies the relationship among program elements.

Reusability

Once a class has been written, created, and debugged, it can be distributed to other
programmers for use in their own programs. This is called *reusability*. It is similar to
the way a library of functions in a procedural language can be incorporated into dif-
ferent programs.

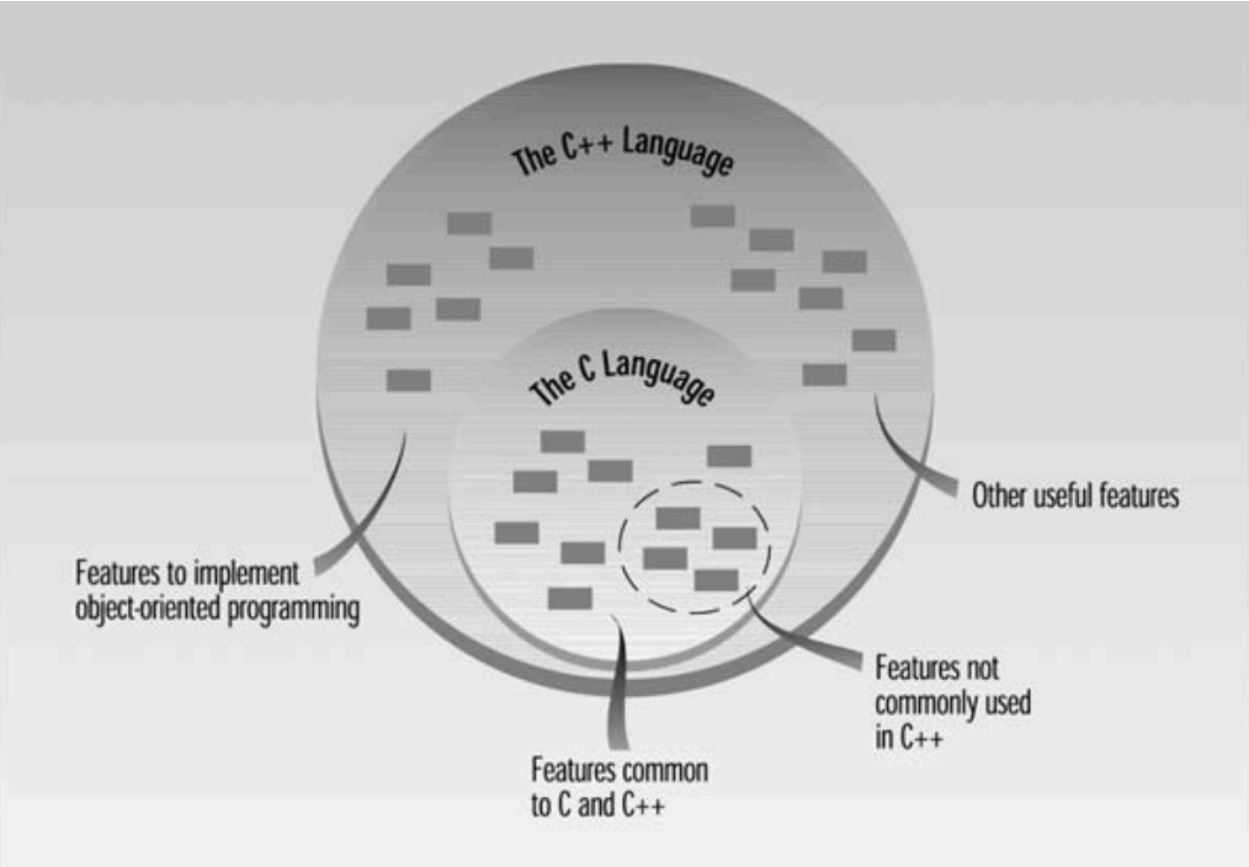
Creating New Data Types - By OOPS approach we can create our own data type using classes and objects

Polymorphism and Overloading

Using operators or functions in different ways, depending on what they are operating on, is
called *polymorphism* (one thing with several distinct forms). When an existing operator, such
as + or =, is given the capability to operate on a new data type, it is said to be *overloaded*.

Overloading is a kind of polymorphism; it is also an important feature of OOP.

C++ is a superset of C language



Summary

- OOP is a way of organizing programs. The emphasis is on the way programs are designed, not on coding details. In particular, OOP programs are organized around objects, which contain both data and functions that act on that data. A class is a tem- plate for a number of objects.

- Inheritance allows a class to be derived from an existing class without modifying it. The derived class has all the data and functions of the parent class, but adds new ones of its own. Inheritance makes possible reusability, or using a class over and over in different programs.
- C++ is a superset of C. It adds to the C language the capability to implement OOP. It also adds a variety of other features. In addition, the emphasis is changed in C++ so that some features common to C, although still available in C++, are seldom used, while others are used far more frequently. The result is a surprisingly different language.
- The Unified Modeling Language (UML) is a standardized way to visualize a program's struc- ture and operation using diagrams.